

List Methods: `ArrayList<String> L = new ArrayList<>(); Node <E> Current = new Node<>();`
`.add("sword");` → adds to end of list ****Note 10 mod 5 = 5****
`.add(0,"katana");` → adds element in index and moves things to right*
`.get(i);` → gets element at index ****Heap → Bubble up/down; Structure in arrays****
`.contains(i)` → returns true if I in list $O(n)$
`.remove(i);` → removes element and moves things to the left (i--)
`.size();` → returns the size of the list

Stack Methods:

`.Push(E item)` → puts an item on top of stack
`.Pop` → removes top item of stack
`.Peek` → lets you look at the top item of stack
`.IsEmpty` → returns true or false

HashMap Methods: (No index for each loop)

`.get(Object Key)`
`.put(K Key, V Value)`
`.remove(Object Key)`
`.isEmpty()`

Queue Methods:

`.Remove` → Removes front
`.Poll` → removes entry at front + give
`.Peek` → Returns entry front w/o remove
`.add(E item)` → adds to end

HashSet Methods:

`.containsAll` → Boolean
`.addAll` → Union
`.retainAll` → Intersection
`.removeAll` → Set difference

In Order (Under) → left, root, Right, **Preorder** (Top Down) → Node, left, Right
Post Order (Bottom up) → Left, Right, Root | Full Tree → 0 or 2 children Perfect
Tree → $2^{\text{height}} - 1$ node Complete Tree → perfect until $n - 1$; pack to left
Merge → Always $O(n \log n)$, **Insertion** → $O(n)$ or $O(n^2)$, **Quick** → $O(n \log n)$ or
 $O(n^2)$ [Left Pointer > Pivot & Right Pointer < Pivot {Swap when matched} Move
pivot when pointers have swapped] Linear Probing = $I + 1$...;
Quad Probing = $I + 1^2$...; Chaining = linkedList at index and add values (ALL FOR COLLISIONS)

Dijkstra's: (PQ)

Init $\text{dist}[v] = \infty$ for all

$\text{Prev}[v] = \text{null}$

$\text{Dist}[\text{start}] = 0$

$\text{Pq.add}(\text{Start}, \text{dist} = 0)$

While pq is not empty:

U = smallest dist node in pq

For each neigh v in u:

$\text{NewDist} = \text{dist}[U] + \text{weight}[U,v]$

If $\text{newDist} < \text{dist}[v]$:

$\text{Dist}[v] = \text{newDist}$

$\text{Prev}[v] = v$

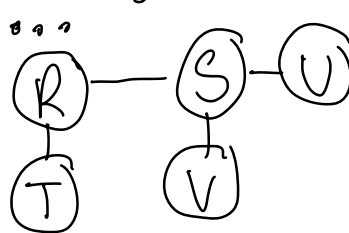
$\text{Pq.add}(v, \text{newDist})$

If ($\text{rootA} == \text{null} \ \&\& \ \text{rootB} == \text{null}$) {
 $\text{B.get}(\text{key});$

Return true; (Test if trees are equal)

Adjacency List:

R: S, +
S: r, U, V



Adjacency Matrix:

	A	B	C	D
A	0	1	1	1
B	1	0	0	0
C	0	1	0	1
D	0	0	0	0

*** 0 = no connect; 1 = connect ***

****Overlap of hash maps****

`HashMap<String,Integer> HM = new HashMap<>();`

`for(String key: A.keySet()){`

`if(B.containsKey(key)){`

`HM.put(key,A.get(key) +`

```

} else if(rootA == null && rootB != null){
    Return false;
} else if(!rootA.item.equals(rootB.item)){ Stack → LIFO; Queue → FIFO
    Return false;
} else { (See recursion and base cases)
    Return equals(rootA.left, rootB.left) && equals(rootA.right,rootB.right)

```

BFS: (Queue)

Visit the start node first

Then all the nodes adjacent to it

Then all nodes that can be reached by a path from the start node **containing two edges**

Then all nodes that can be reached by a path from the start node **containing three edges**

And onward

****For Each loop format****

DFS: (Stack/Recursion)

for (DataType item : Collection) {

Start at vertex

Visit it

Choose adjacent one to visit

Then choose a vertex adjacent to that vertex to visit (ABC's?)

And so, on until you can go no further

Then back up and see whether a new vertex can be found

Huffman Encoding → PQ of the smallest freq; Build up from the right (Left/Right!!)

Heaps: ("Heapify" in subtrees min or max)

ArrayList vs linkedList

get/set → O(1) vs O(n)

add front → O(n) vs O(1)

add middle → O(n) vs O(n)

add back → O(1) vs O(1)

Removing From BST Tree:

If 2 child go to *left subtree*; pick *largest element* there

```

Public Static <E> LinkedList createList(TreeNode<E> root){
    LinkedList<E> list = new LinkedList<E>();
    if(root == null){
        return list;
    }
    list.add(root.data);
    list.addAll(createList(root.left);
    list.adedAll(createList(root.right); }

```